

ANIMAL-CARE VETERINARY CLINIC

420-DW4-AS Web Server Applications Development II



NOVEMBER 21, 2025

Rafat-Ahmed Islam (2311953), Shaquille Osborne Neil (2330041), Kevin Alinazar (2332512), Alvaro Eloy Limaymanta Soria (2311145)

Presented to

Houria Houmel

Table of Contents

Part I – Information System Analysis & Design 2
1-System Analysis by Rafat-Ahmed Islam 2
1.1- Identify the system objects to be automated. 2
1.2- Identify the characteristics of each object and their functionalities 3
2- Design (UML Disgrams) 4
2.1- Use Case Diagrams by Shaquille Osborne Neil 4
2.2- Class Diagram by Kevin Alinazar 8
2.3- Activity Diagram by Alvaro Eloy Limaymanta Soria 9
2.4- Sequence Diagram by Rafat-Ahmed Islam 12
Part II – Design & Database Implementation 13
1-Entity-Relationship Diagram – ERD 13
1.1 Identifying Entities and Their Characteristics 13
1.2 Relationship & Cardinalities 13
1.3 Entity-Relationship Diagram 14
2 – Relational Data Model – RDM 15
2.1 Identify parent tables and association tables 15
2.2 Define Domain-Level Constraints (Column-Level Constraints) 19
2.3 Define table constraints and relationship constraints between tables. 21
2.4 Relational Data Model 24
3 - Implementation of the Database Using Microsoft SQL Server 2019/2022 25
3.1 Overview 25
3.2 Database Creation & Setup 25
3.3 Views 25
3.4 Stored Procedures 28
3.5 Triggers 30
3.6 Indexes 33
3.7 Testing & Validation 35
3.8 Summary and Evaluation of Database Implementation 36

Part I – Information System Analysis & Design

1- System Analysis by Rafat-Ahmed Islam

1.1- Identify the system objects to be automated.

- **User**: Represents any person authorized to access the system (administrator, receptionist, veterinarian).
- **Role**: Represents the level of access permissions granted to a user.
    1. **Administrator:** Full access to all system functionalities.
    2. **Receptionist:** Manages owners, animals, and appointments.
    3. **Veterinarian:** Views schedules, manages their own appointments, and records medical histories.
- **Veterinarian** (Role): Can view schedules, manage their appointments, and record medical histories.
- **Owner:** Represents the clinic's client.
- **Animal:** Represents the animal patient cared for by the clinic.
- **Schedule:** Represents the availability time slots of a veterinarian.
- **Appointment:** Represents a planned consultation between an owner (through their animal) and a veterinarian.
- **MedicalHistory:** Represents the animal's medical record, keeping track of all treatments, diagnoses, and procedures performed.
- **Notification:** Represents the system for automatic alerts and reminders.

1.2- Identify the characteristics of each object and their functionalities

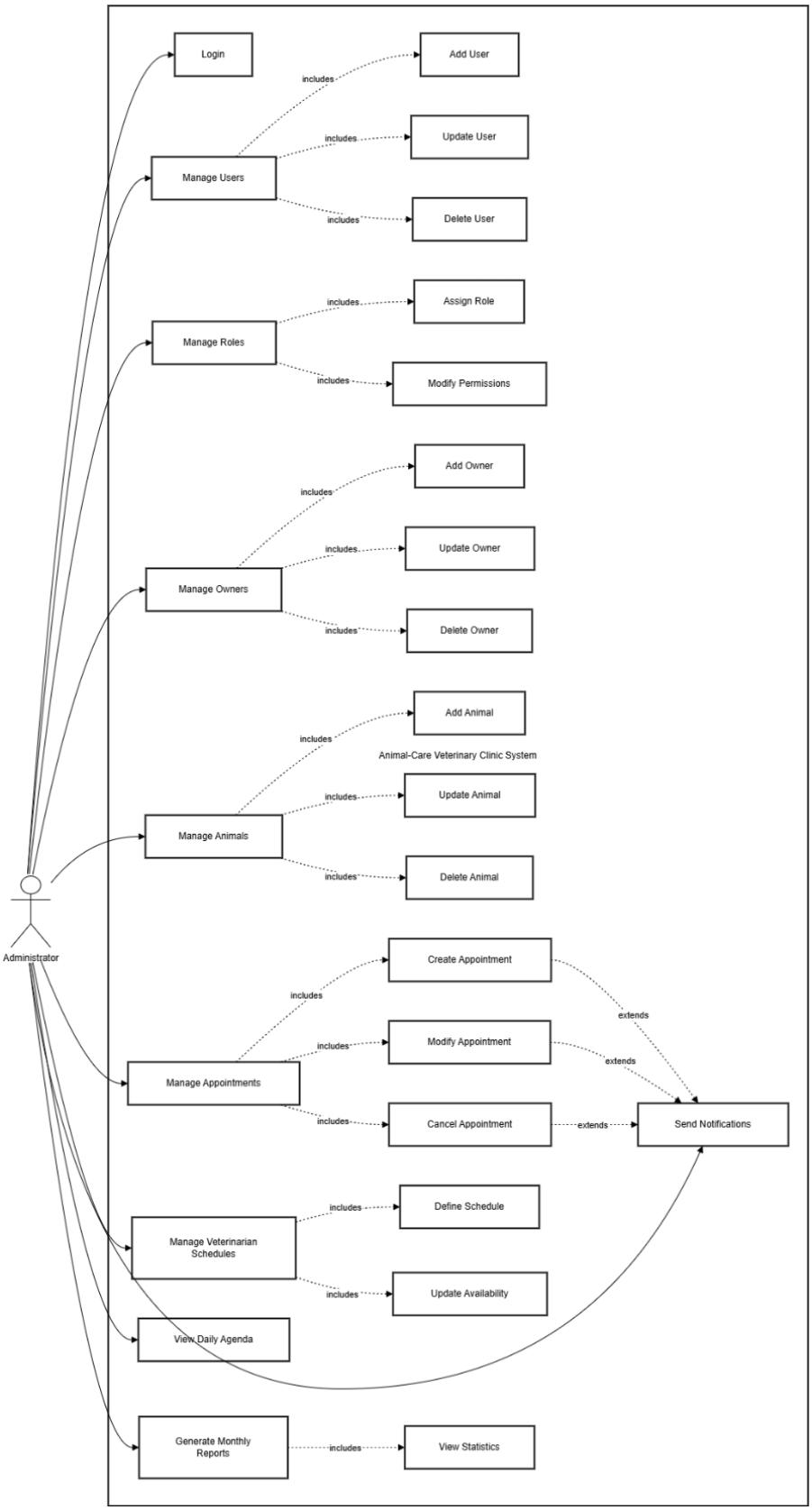
Object	Attributes	Main Functions
Owner	IdOwner, LastName, FirstName, Phone, Email, Address	- Register a new owner- Update owner information- View the list of their animals- Delete a client record
Animal	IdAnimal, Name, Species, Breed, Sex, BirthDate, Weight	- Add an animal to the owner's record- View information and medical history- Modify data- Delete an animal
Veterinarian	IdVet, LastName, FirstName, Specialty, Phone, Email	- Manage availability- View upcoming appointments- Add notes or diagnoses to medical records- Update profile
Schedule / Availability	IdSchedule, IdVet (FK), Date, StartTime, EndTime, Status (Free / Busy)	- Define work time slots- Update veterinarian availability- Prevent double-booking of the same slot
Appointment	IdAppointment, IdVet (FK), IdAnimal (FK), StartDateTime, EndDateTime, Purpose, Status (Planned / Canceled / Completed)	- Create an appointment based on veterinarian availability- Modify or cancel an appointment- Display appointments for a given day- Link appointment ↔ animal ↔ veterinarian
User	IdUser, Login, Password, IdRole (FK), Active (Yes / No)	- Authenticate system users- Control access according to role (Admin, Receptionist, Veterinarian)- Manage account status (active/inactive)
Role	IdRole, RoleName (Administrator / Receptionist / Veterinarian)	- Define permission levels- Manage access rights to modules- Assign each user a role
Medical History / Record	IdHistory, IdAnimal (FK), ConsultationDate, Observations, Diagnosis, Treatment, IdVet (FK)	- Record each medical visit- View the complete history of an animal- Add or modify a diagnosis- Track treatments administered
Notification	IdNotification, IdAppointment (FK), NotificationType (Reminder / Cancellation / Change), SendDate, Channel (Email / SMS), Status (Sent / Pending)	- Automatically send appointment reminders- Notify the client of cancellations or changes- Manage notification status
View: MonthlyReport	Month, Year, NumberOfAppointments, NumberCanceled, NumberOfActiveVets, OccupancyRate	- Generate statistical reports- Visualize monthly activity trends-

Object	Attributes	Main Functions
		Assist administrators in decision-making
View: DailyAgenda	Date, VeterinarianName, StartTime, EndTime, AnimalName, OwnerName, Status	- Display daily appointments- Facilitate daily planning- Provide a real-time view of consultations

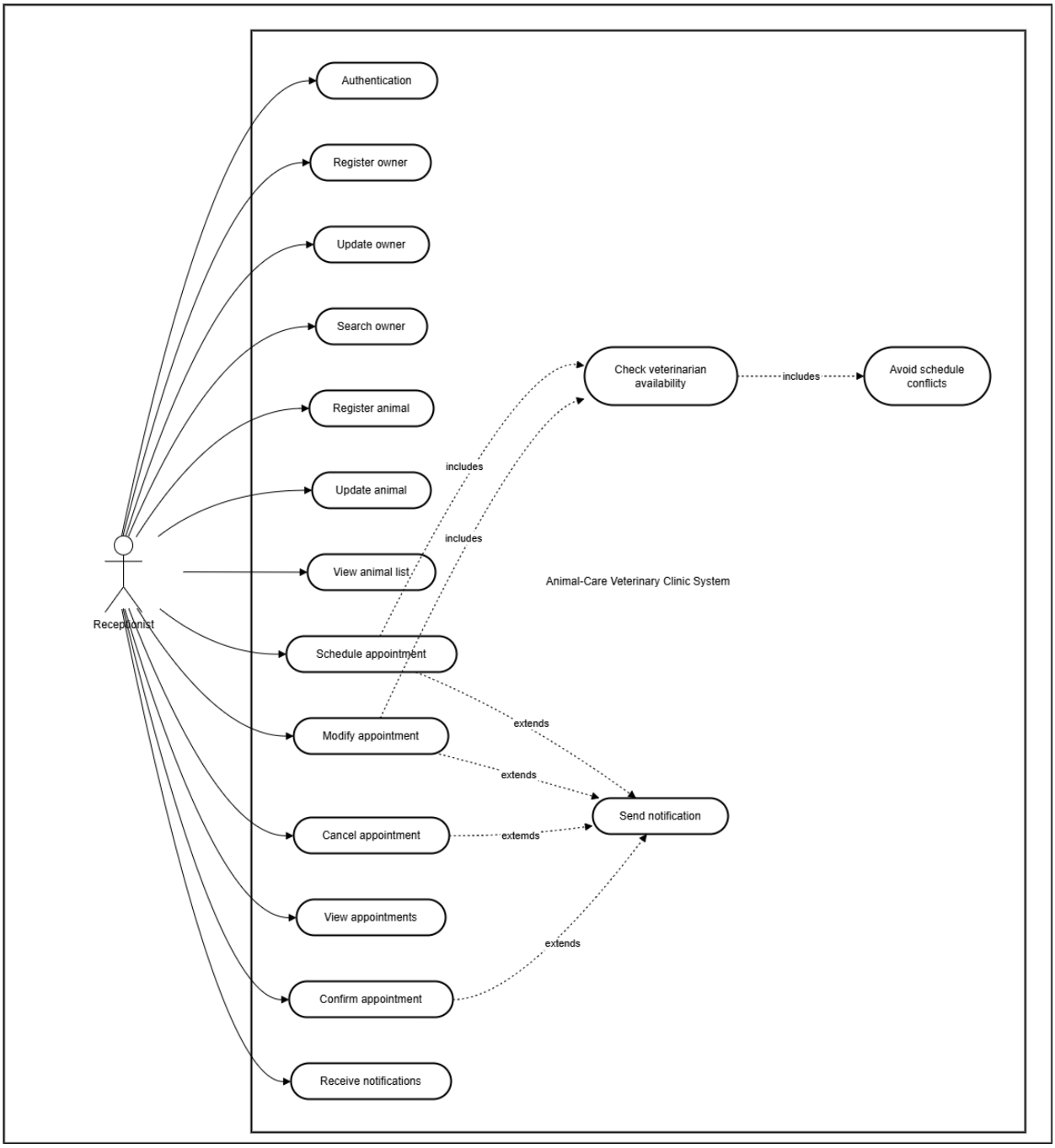
2- Design (UML Disgrams)

2.1- Use Case Diagrams by Shaquille Osborne Neil

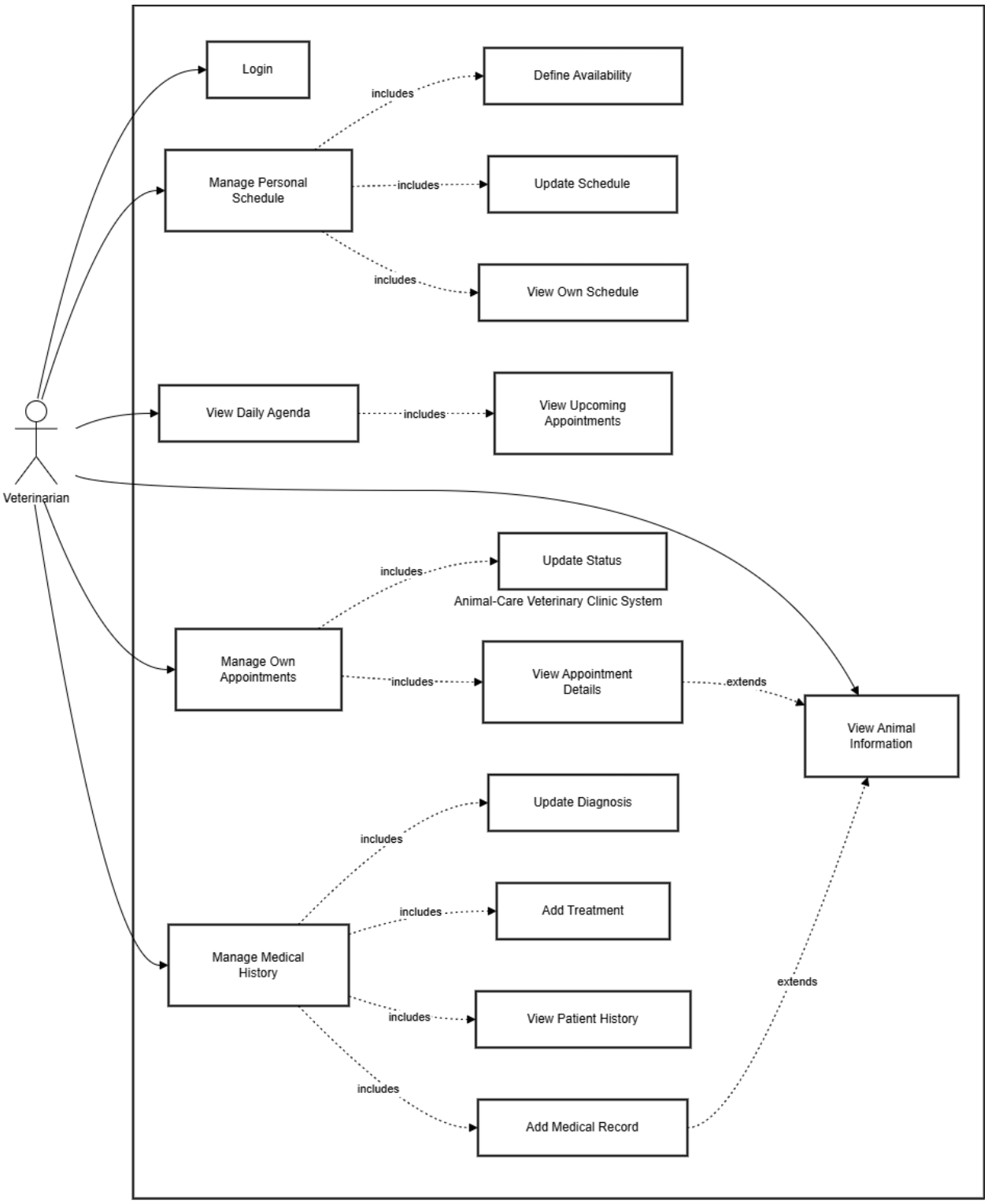
Administrator Use Case Diagram



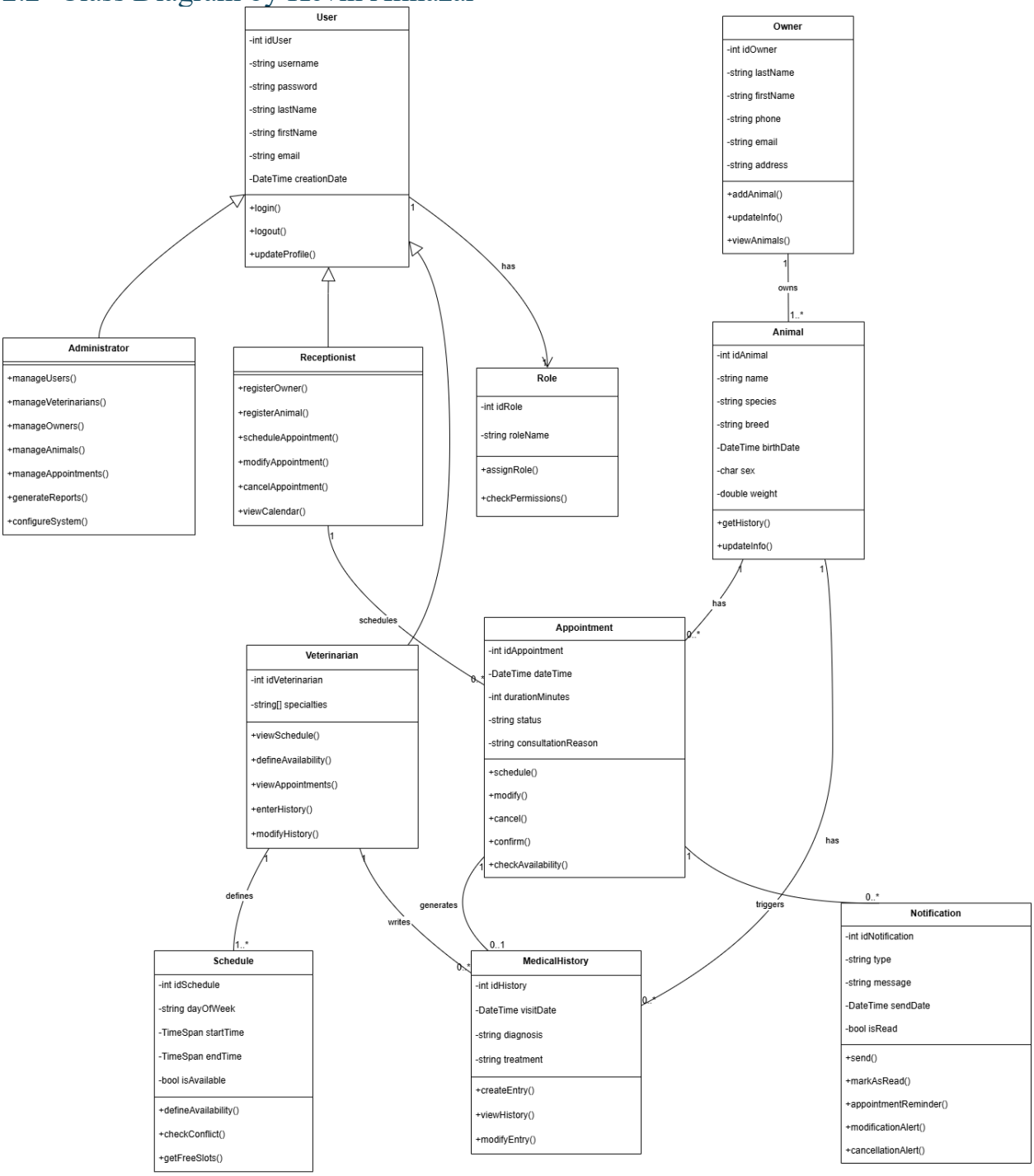
Receptionist Use Case Diagram



Veterinary Use case diagram

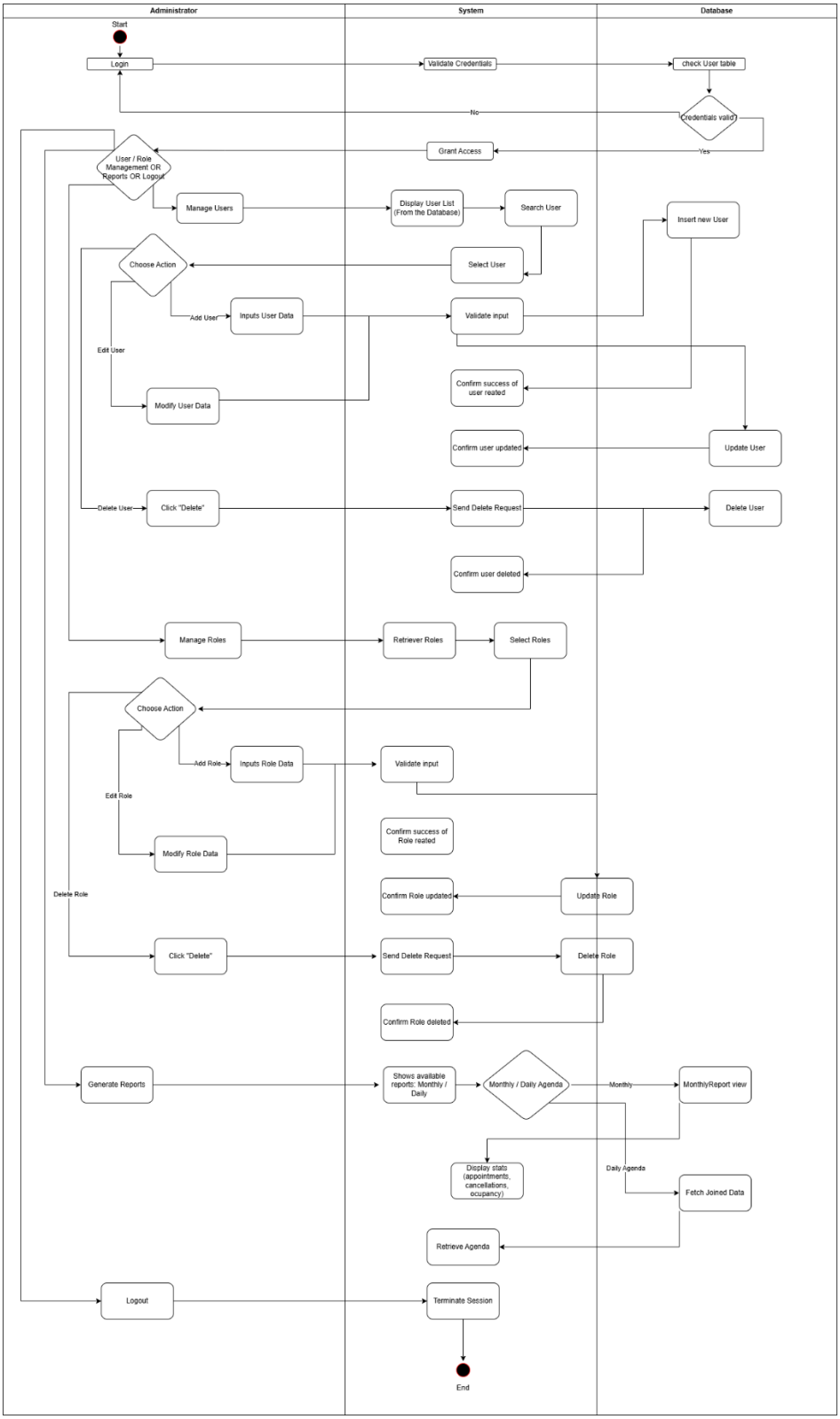


2.2- Class Diagram by Kevin Alinazar

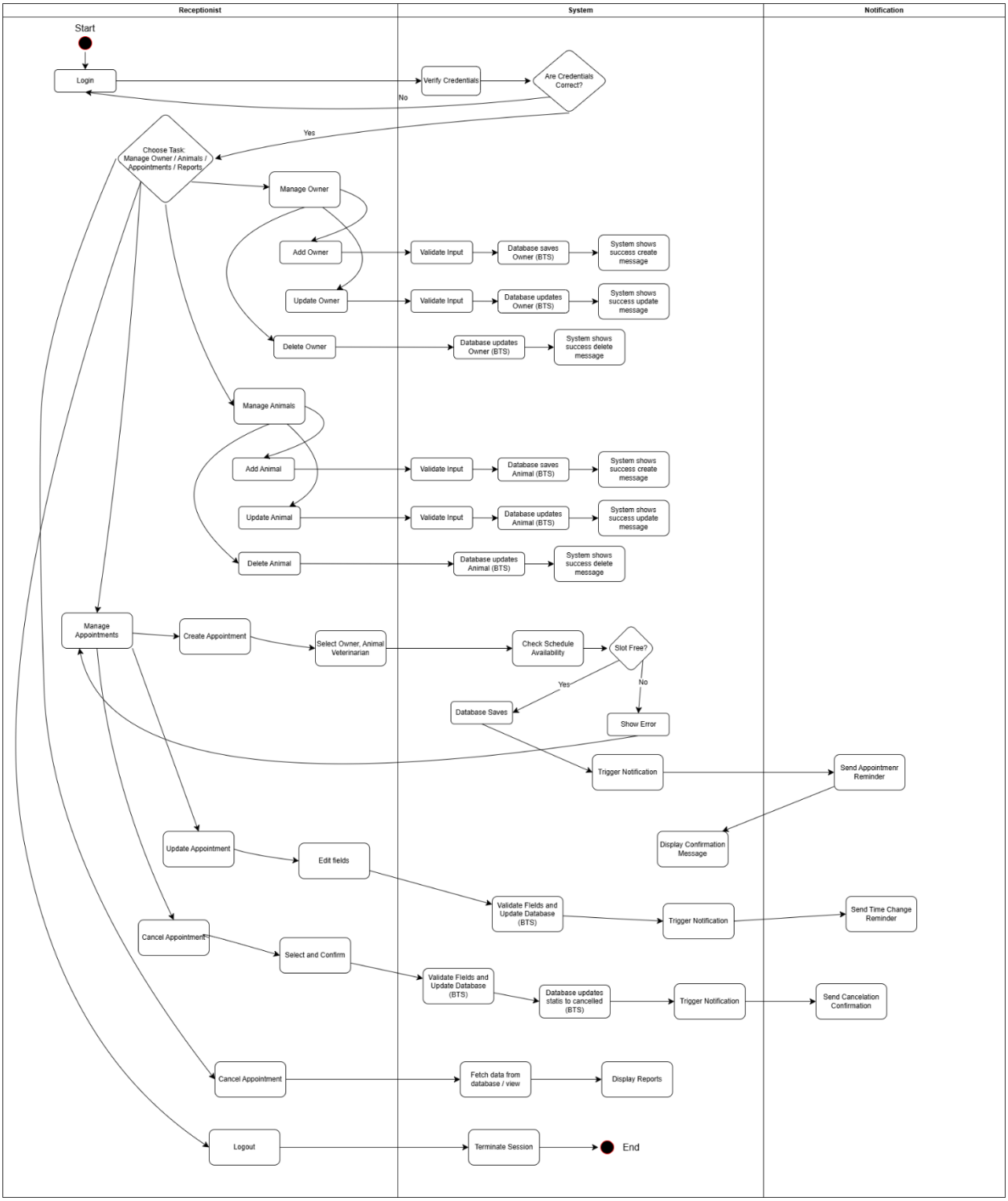


2.3- Activity Diagram by Alvaro Eloy Limaymanta Soria

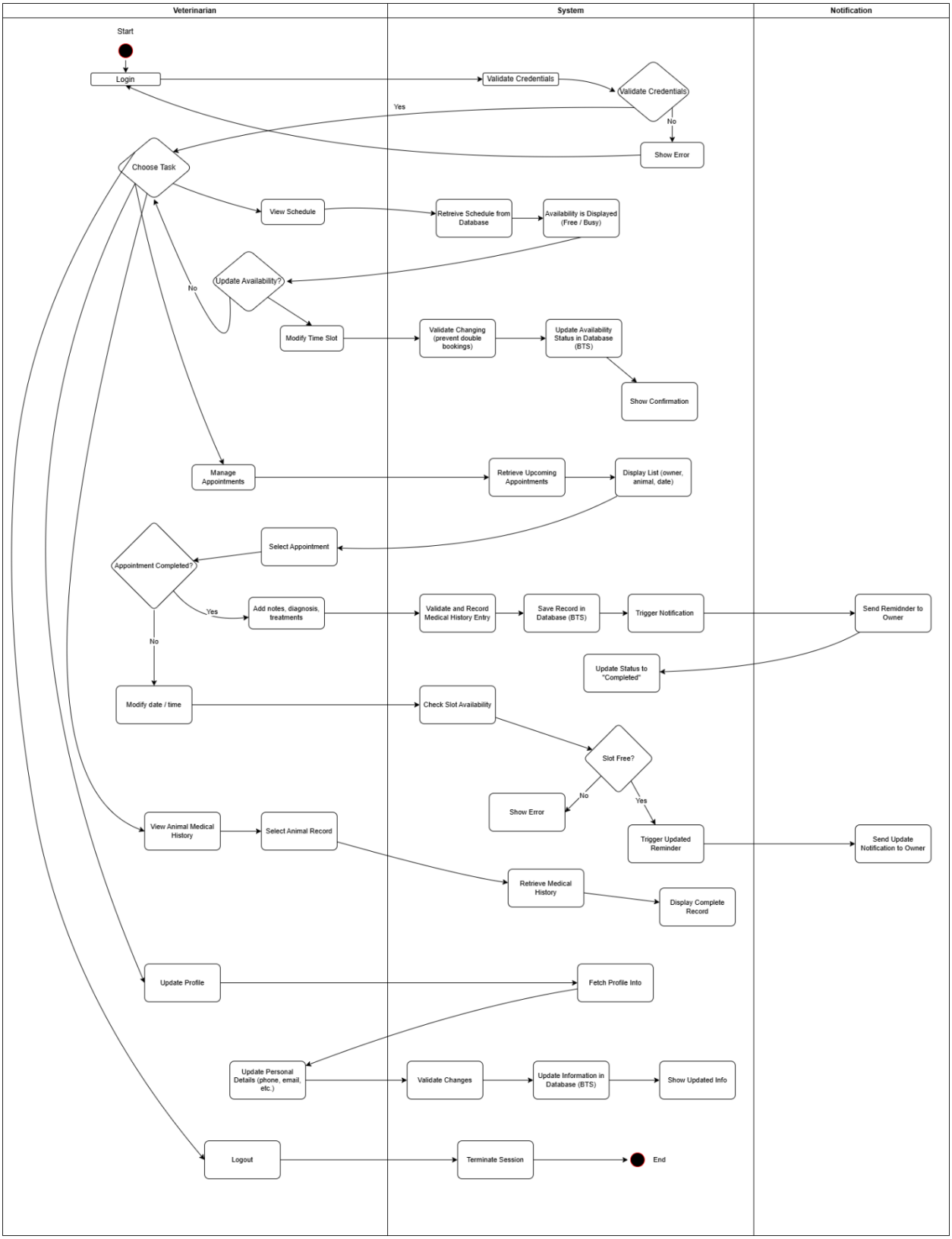
Activity Diagram Administration



Activity Diagram Reception



Activity Diagram Veterinarian



2.4- Sequence Diagram by Rafat-Ahmed Islam

Administrator Sequence Diagram



sequence_diagram_administrator.drawio.p

Note: Double click the PDF file to see Administrator Sequence Diagram in full screen.

Reception Sequence Diagram



sequence_diagram_reception.drawio.pdf

Note: Double click the PDF file to see Reception Sequence Diagram in full screen.

Veterinary Sequence Diagram



sequence_diagram_veterinary.drawio.pdf

Note: Double click the PDF to see Veterinary Sequence Diagram in full screen.

Note: You can only double-click the PDF files in the Word document. If you are viewing the PDF version of this document, you will need to open *Animal_care_veterinary_clinic.docx* to check the sequence, or you can access the folder that contains the images of the sequence diagrams along with all the other included diagrams.

Part II – Design & Database Implementation

1- Entity-Relationship Diagram – ERD

1.1 Identifying Entities and Their Characteristics

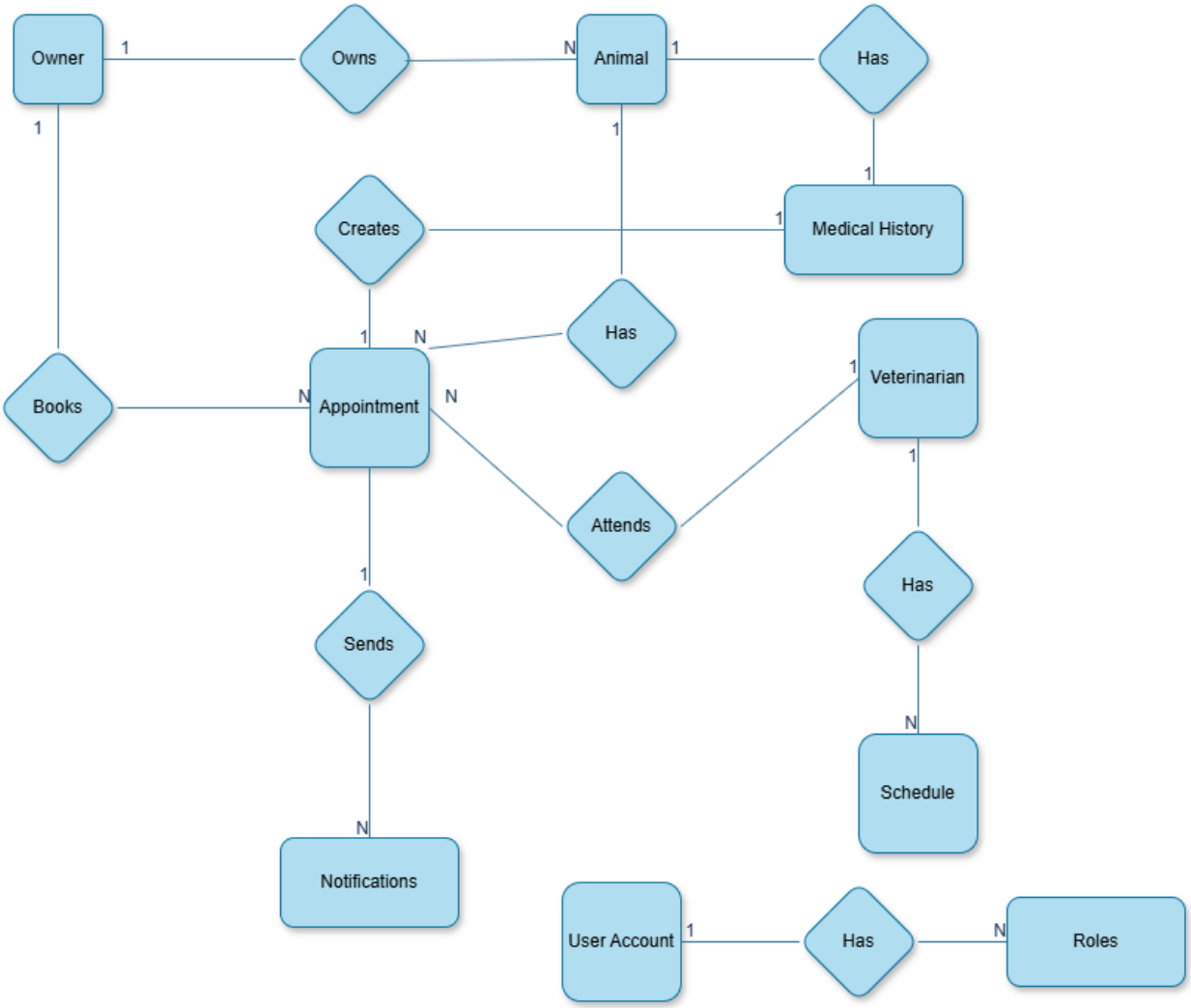
The main entities identified in the *Animal-Care Veterinary Clinic* system are:

- **Owner** --> stores the owner's personal information (name, phone, email, address).
- **Animal** --> represents each pet; includes species, breed, age, and weight.
- **Veterinarian** --> contains details about each vet and their specialty.
- **Schedule** --> defines each veterinarian's daily availability.
- **Appointment** --> links a vet and an animal for a consultation with start / end time.
- **Medical History** --> records each animal's consultations, diagnoses, and treatments.
- **Notification** --> tracks appointment reminders, cancellations, or changes.
- **UserAccount** --> stores login credentials and activation status for system users.
- **Role** --> defines each user's permission level (Administrator, Veterinarian, Receptionist).

1.2 Relationship & Cardinalities

- One **Owner** can have many **Animals** (1 --> N).
- One **Veterinarian** can have many **Schedules** and **Appointments** (1 --> N).
- One **Animal** can have many **Appointments** and **Medical History** records (1 --> N).
- Each **Appointment** can generate many **Notifications** (1 --> N).
- Each **Role** can be assigned to many **UserAccounts** (1 --> N).

1.3 Entity-Relationship Diagram



2 – Relational Data Model – RDM

2.1 Identify parent tables and association tables

Parent Tables

Parent tables are independent entities that other tables depend on through foreign keys.
They contain core information and are not dependent on any other table.

Parent Table	Description
Role	Defines user permission levels such as Administrator, Veterinarian, and Receptionist.
Owner	Stores client (pet owner) information such as name, contact, and address.
Veterinarian	Contains information about each veterinarian and their specialties.

Child / Association Tables

These tables depend on parent tables and use **foreign keys** to define relationships.

Child Table	Parent Reference	Description
UserAccount	FK → Role(RoleId)	Each user belongs to a specific role.
Animal	FK → Owner(OwnerId)	Each animal is linked to its owner.
Schedule	FK → Veterinarian(VetId)	Defines available work slots for each vet.
Appointment	FK → Veterinarian(VetId) and Animal(AnimalId)	Connects a vet with an animal for a scheduled visit.
MedicalHistory	FK → Animal(AnimalId), Veterinarian(VetId)	Stores past consultations and treatments for each animal.
Notification	FK → Appointment(AppointmentId)	Tracks appointment reminders, cancellations, or updates.

Columns and Attribute Types

Table Name	Column Name	Data Type	Constraints / Notes
Role	RoleId	INT IDENTITY(1,1)	Primary Key
	RoleName	NVARCHAR(50)	NOT NULL
UserAccount	UserId	INT IDENTITY(1,1)	Primary Key
	UserLogin	NVARCHAR(100)	UNIQUE, NOT NULL
	UserPassword	NVARCHAR(255)	NOT NULL
	RoleId	INT	Foreign Key → Role(RoleId)
	UserActive	BIT	DEFAULT 1
Owner	OwnerId	INT IDENTITY(1,1)	Primary Key
	OwnerLastName	NVARCHAR(50)	
	OwnerFirstName	NVARCHAR(50)	
	OwnerPhone	NVARCHAR(20)	
	OwnerEmail	NVARCHAR(100)	
	OwnerAddress	NVARCHAR(255)	
Veterinarian	VetId	INT IDENTITY(1,1)	Primary Key
	VetLastName	NVARCHAR(50)	
	VetFirstName	NVARCHAR(50)	
	VetSpecialty	NVARCHAR(100)	
	VetPhone	NVARCHAR(20)	
	VetEmail	NVARCHAR(100)	
Animal	AnimalId	INT IDENTITY(1,1)	Primary Key
	AnimalName	NVARCHAR(50)	
	Species	NVARCHAR(50)	

Table Name	Column Name	Data Type	Constraints / Notes
	Breed	NVARCHAR(50)	
	Sex	CHAR(1)	
	BirthDate	DATE	
	AnimalWeight	DECIMAL(5,2)	
	OwnerId	INT	Foreign Key → Owner(OwnerId)
Schedule	ScheduleId	INT IDENTITY(1,1)	Primary Key
	VetId	INT	Foreign Key → Veterinarian(VetId)
	ScheduleDate	DATE	NOT NULL
	StartTime	TIME	NOT NULL
	EndTime	TIME	NOT NULL
	ScheduleStatus	NVARCHAR(10)	CHECK (‘Free’, ‘Busy’)
Appointment	AppointmentId	INT IDENTITY(1,1)	Primary Key
	VetId	INT	Foreign Key → Veterinarian(VetId)
	AnimalId	INT	Foreign Key → Animal(AnimalId)
	StartDateTime	DATETIME	NOT NULL
	EndDateTime	DATETIME	NOT NULL
	Purpose	NVARCHAR(255)	
	AppointmentStatus	NVARCHAR(15)	CHECK (‘Planned’, ‘Canceled’, ‘Completed’)
MedicalHistory	HistoryId	INT IDENTITY(1,1)	Primary Key
	AnimalId	INT	Foreign Key → Animal(AnimalId)
	ConsultationDate	DATE	NOT NULL
	Observations	NVARCHAR(MAX)	

Table Name	Column Name	Data Type	Constraints / Notes
	Diagnosis	NVARCHAR(MAX)	
	Treatment	NVARCHAR(MAX)	
	VetId	INT	Foreign Key → Veterinarian(VetId)
Notification	NotificationId	INT IDENTITY(1,1)	Primary Key
	AppointmentId	INT	Foreign Key → Appointment(AppointmentId)
	NotificationType	NVARCHAR(20)	CHECK (‘Reminder’, ‘Cancellation’, ‘Change’)
	SendDate	DATETIME	
	Channel	NVARCHAR(10)	CHECK (‘Email’, ‘SMS’)
	NotificationStatus	NVARCHAR(10)	CHECK (‘Sent’, ‘Pending’)

2.2 Define Domain-Level Constraints (Column-Level Constraints)

Each column in the database is restricted by appropriate domain-level constraints to ensure data validity. For example, enumerated attributes such as **AppointmentStatus** and **ScheduleStatus** use **CHECK** constraints to limit acceptable values. Columns like **UserLogin** and **UserPassword** are defined as **NOT NULL** to guarantee that required information is always present. Additionally, **DEFAULT** constraints (For example: **UserActive = 1**) automatically assign values when none are provided, maintaining data consistency.

Table	Column	Constraint Type	Description / Rule
Role	RoleName	NOT NULL	Every role must have a name.
UserAccount	UserLogin	UNIQUE, NOT NULL	Each user login must be unique and cannot be empty.
	UserPassword	NOT NULL	Passwords must be provided for all users.
	UserActive	DEFAULT 1	Defaults to 1 (active) unless specified otherwise.
Schedule	ScheduleStatus	CHECK (ScheduleStatus IN ('Free','Busy'))	Limits the schedule state to only “Free” or “Busy.”
Appointment	AppointmentStatus	CHECK (AppointmentStatus IN ('Planned','Canceled','Completed'))	Restricts possible appointment statuses.
	StartDateTime, EndDateTime	NOT NULL	Appointment time values must be defined.

Table	Column	Constraint Type	Description / Rule
Notification	NotificationType	CHECK (NotificationType IN ('Reminder','Cancellation','Change'))	Only allows three valid notification types.
	Channel	CHECK (Channel IN ('Email','SMS'))	Notification channel limited to Email or SMS.
	NotificationStatus	CHECK (NotificationStatus IN ('Sent','Pending'))	Restricts notification delivery state.
MedicalHistory	ConsultationDate	NOT NULL	Every record must include a consultation date.
Animal	Sex	CHECK (Sex IN ('M','F')) *(optional enhancement)*	Ensures the sex value is only M or F.
	AnimalWeight	DECIMAL(5,2)	Limits the number to two decimal places and reasonable weight range.
All ID columns	Primary Key (IDENTITY(1,1))	Auto-increments and uniquely identifies each record.	
All Foreign Keys	FOREIGN KEY constraints	Enforce referential integrity between related tables.	

2.3 Define table constraints and relationship constraints between tables.

Each table in the AnimalCareDB includes primary keys to uniquely identify records and foreign keys to maintain referential integrity between related entities. Table-level constraints such as **CHECK**, **DEFAULT**, and **UNIQUE** ensure that data stored in the system follows valid business rules. Relationship constraints enforce proper linkage between parent and child tables. For example, an Appointment must always reference an existing Veterinarian and Animal. Together, these constraints guarantee the overall consistency and reliability of the database.

A. Table Constraints

Table constraints ensure **data integrity at the table level**, by enforcing rules such as **primary keys**, **unique values**, and **check limits** within each table.

Table	Constraint Type	Definition / Purpose
Role	PRIMARY KEY (RoleId)	Ensures each role record is unique.
UserAccount	PRIMARY KEY (UserId)	Uniquely identifies each user.
	UNIQUE (UserLogin)	Prevents duplicate login names.
	DEFAULT (UserActive = 1)	Sets users as active by default.
Owner	PRIMARY KEY (OwnerId)	Ensures each owner record is unique.
Veterinarian	PRIMARY KEY (VetId)	Identifies each veterinarian uniquely.
Animal	PRIMARY KEY (AnimalId)	Ensures each animal record is unique.
	CHECK (Sex IN ('M','F'))	Restricts valid values for gender.
Schedule	PRIMARY KEY (ScheduleId)	Identifies each schedule entry uniquely.

Table	Constraint Type	Definition / Purpose
	CHECK (ScheduleStatus IN ('Free','Busy'))	Restricts valid schedule statuses.
Appointment	PRIMARY KEY (AppointmentId)	Ensures every appointment has a unique ID.
	CHECK (AppointmentStatus IN ('Planned','Canceled','Completed'))	Restricts allowed appointment states.
MedicalHistory	PRIMARY KEY (HistoryId)	Ensures every medical record is unique.
Notification	PRIMARY KEY (NotificationId)	Ensures each notification is unique.
	CHECK (NotificationStatus IN ('Sent','Pending'))	Restricts delivery state values.

B. Relationship Constraints

Relationship constraints are implemented through **foreign keys (FKs)**.
They enforce **referential integrity** between parent and child tables to ensure that related records exist before insertion and that deletions or updates are consistent.

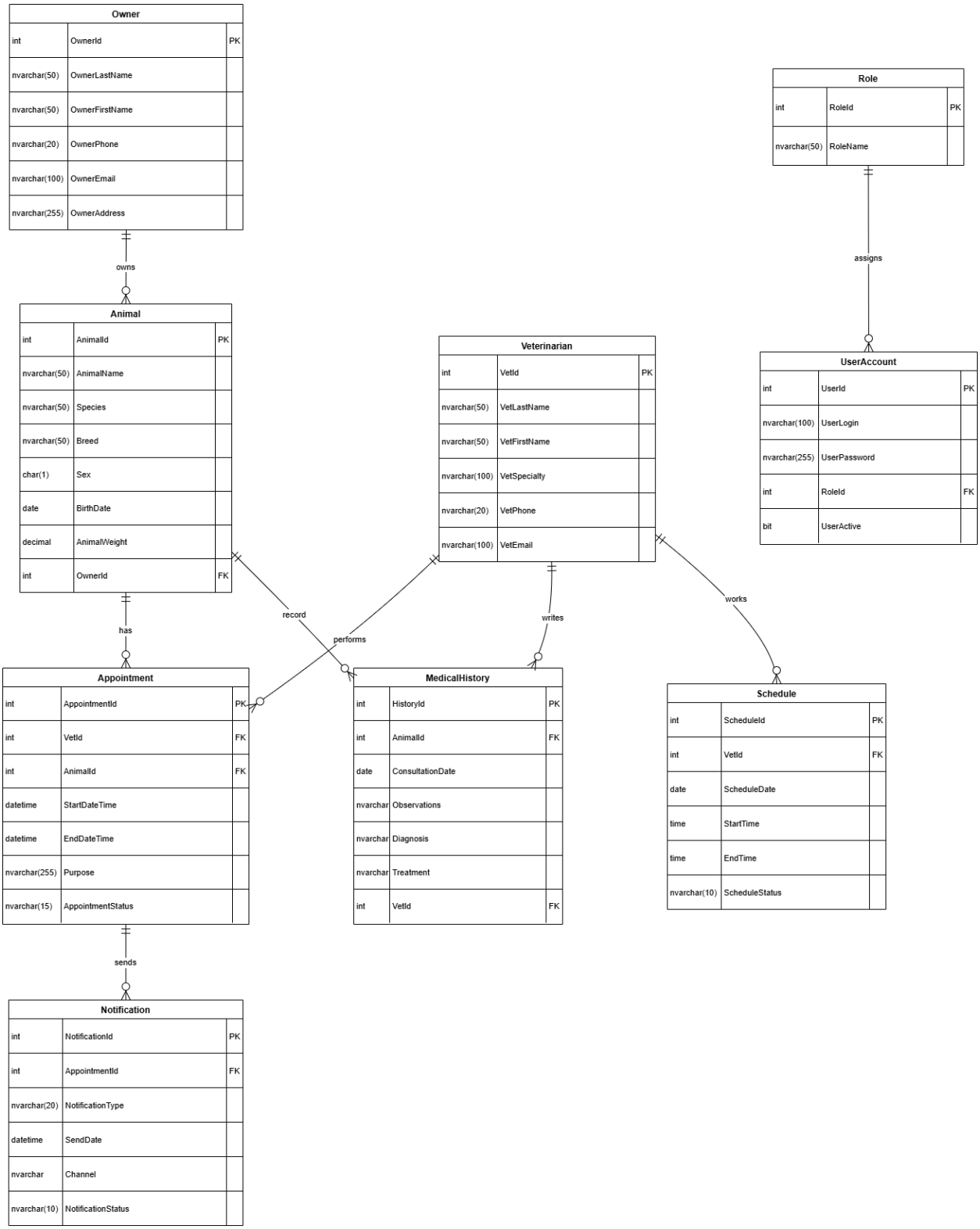
Child Table	Foreign Key	References (Parent Table)	Relationship Type	Explanation
UserAccount	RoleId	Role(RoleId)	1 → N	Each user belongs to one role; one role can have many users.
Animal	OwnerId	Owner(OwnerId)	1 → N	Each animal belongs to one owner; an owner can have multiple animals.
Schedule	VetId	Veterinarian(VetId)	1 → N	Each vet can have multiple scheduled time slots.

Child Table	Foreign Key	References (Parent Table)	Relationship Type	Explanation
Appointment	VetId	Veterinarian(VetId)	1 → N	Each vet can perform many appointments.
	AnimalId	Animal(AnimalId)	1 → N	Each animal can have multiple appointments.
MedicalHistory	VetId	Veterinarian(VetId)	1 → N	Each vet can have multiple medical history records.
	AnimalId	Animal(AnimalId)	1 → N	Each animal can have multiple history records.
Notification	AppointmentId	Appointment(AppointmentId)	1 → N	Each appointment can trigger multiple notifications.

C. Referential Integrity Rules

- **Foreign key relationships** ensure that child records cannot exist without their parent. (e.g., an Appointment cannot exist without a valid Veterinarian and Animal.)
- **Cascade behaviors** (if implemented later) can be used to automatically remove dependent records when a parent is deleted.
- **CHECK** and **DEFAULT** constraints at the table level help maintain valid and consistent entries across all relationships.

2.4 Relational Data Model



3 - Implementation of the Database Using Microsoft SQL Server 2019/2022

3.1 Overview

The database for the **Animal-Care Veterinary Clinic** system was implemented using **Microsoft SQL Server 2022**.
The implementation phase focused on translating the relational data model into an executable database schema using **Transact-SQL (T-SQL)**.
This included the creation of all tables, relationships, constraints, views, stored procedures, triggers, and insertion of sample data for testing.

3.2 Database Creation & Setup

The database was created using the following command:

```
CREATE DATABASE AnimalCareDB;

USE AnimalCareDB;
```

- All tables, primary keys, foreign keys, and constraints were defined exactly as described in the relational data model (RDM).
- The script includes IF OBJECT_ID checks to safely drop and recreate objects when necessary.
- Sample data was inserted into each table for testing (one record per entity).

Example:

```
INSERT INTO Veterinarian (VetLastName, VetFirstName, VetSpecialty, VetPhone, VetEmail)

VALUES ('Shaqille', 'Osborne', 'General Care', '514-098-9876', 'shaquille@animalcare.com');
```

3.3 Views

Two main **views** were created to summarize and simplify the retrieval of frequently used data.

a) MonthlyReport View

This view provides statistical summaries for administrative reporting:

- Number of appointments and cancellations per month
- Total number of active veterinarians
- Monthly occupancy rate (percentage of used schedule slots)

```
CREATE VIEW MonthlyReport AS
SELECT
  MONTH(A.StartDateTime) AS [Month],
  YEAR(A.StartDateTime) AS [Year],
  COUNT(A.AppointmentId) AS NumberOfAppointments,
  SUM(CASE WHEN A.AppointmentStatus = 'Canceled' THEN 1 ELSE 0 END) AS NumberCanceled,
  ISNULL(
    (SELECT COUNT(DISTINCT V.VetId)
     FROM Veterinarian V
     INNER JOIN Schedule S ON V.VetId = S.VetId
     WHERE S.ScheduleStatus IN ('Free','Busy')), 0
  ) AS NumberOfActiveVets,
  ISNULL(
    CAST(
      (
        SUM(CASE WHEN A.AppointmentStatus IN ('Planned','Completed') THEN 1 ELSE 0 END) *
100.0
        /
        NULLIF((SELECT COUNT(*) FROM Schedule S WHERE S.ScheduleStatus IN ('Free','Busy')),0)
      ) AS DECIMAL(5,2)
    ), 0
  ) AS OccupancyRate
FROM Appointment A
GROUP BY YEAR(A.StartDateTime), MONTH(A.StartDateTime);
```

Purpose: Helps administrators track monthly clinic activity and vet workload.

b) DailyAgenda View

This view consolidates all daily appointments into an easy-to-read format for veterinarians and receptionists.

```
CREATE VIEW DailyAgenda AS
SELECT
  CAST(a.StartDateTime AS DATE) AS [Date],
  v.VetLastName + ' ' + v.VetFirstName AS VeterinarianName,
  CONVERT(TIME, a.StartDateTime) AS StartTime,
  CONVERT(TIME, a.EndDateTime) AS EndTime,
  an.AnimalName AS AnimalName,
  o.OwnerLastName + ' ' + o.OwnerFirstName AS OwnerName,
  a.AppointmentStatus
FROM Appointment a
JOIN Veterinarian v ON a.VetId = v.VetId
JOIN Animal an ON a.AnimalId = an.AnimalId
JOIN Owner o ON an.OwnerId = o.OwnerId;
```

Purpose: Displays daily consultation details for better scheduling and coordination.

3.4 Stored Procedures

Stored procedures were created to handle repetitive actions and automate logic at the database level.

a) sp_AddAppointment

Adds a new appointment while ensuring that there is no scheduling conflict for the selected veterinarian.

```
CREATE PROCEDURE sp_AddAppointment
  @Vet INT,
  @Animal INT,
  @Start DATETIME,
  @End DATETIME,
  @Purpose NVARCHAR(255)
AS
BEGIN
  IF EXISTS (SELECT 1 FROM Appointment WHERE VetId=@Vet AND
AppointmentStatus<>'Canceled' AND @Start < EndDateTime AND @End > StartDateTime)
  BEGIN
    RAISERROR('Conflict: Veterinarian already booked.',16,1);
    RETURN;
  END
  INSERT INTO Appointment(VetId, AnimalId, StartDateTime, EndDateTime, Purpose,
AppointmentStatus)
  VALUES(@Vet,@Animal,@Start,@End,@Purpose,'Planned');
END;
```

Purpose: Prevents double-booking and maintains consistent scheduling.

b) sp_UpdateAppointmentStatus

Updates the appointment's status (Planned, Canceled, Completed).

```
CREATE PROCEDURE sp_UpdateAppointmentStatus
  @Appointment INT,
  @NewStatus NVARCHAR(15)
AS
BEGIN
  UPDATE Appointment SET AppointmentStatus=@NewStatus WHERE
AppointmentId=@Appointment;
END;
```

Purpose: Allows administrators or receptionists to manage appointment statuses efficiently.

c) sp_GetVetAgenda

Retrieves a veterinarian's daily appointments from the DailyAgenda view.

```
CREATE PROCEDURE sp_GetVetAgenda
  @Vet INT,
  @TargetDate DATE
AS
BEGIN
  SELECT * FROM DailyAgenda WHERE VeterinarianName IN (
    SELECT VetLastName + ' ' + VetFirstName FROM Veterinarian WHERE VetId=@Vet
  ) AND [Date]=@TargetDate;
END;
```

Purpose: Provides quick access to a veterinarian's daily schedule.

3.5 Triggers

Three database triggers were implemented to automatically enforce business rules and ensure that key operations occur without requiring manual intervention. These triggers help maintain data integrity, automate workflow steps, and enforce scheduling logic.

a) trg_PreventDoubleBooking

This trigger prevents the creation of an appointment if the veterinarian is already booked during the same time period.

```
CREATE TRIGGER trg_PreventDoubleBooking
ON Appointment
AFTER INSERT
AS
BEGIN
  SET NOCOUNT ON;

  IF EXISTS (
    SELECT 1
    FROM Appointment A
    JOIN inserted I ON A.VetId = I.VetId
    WHERE A.AppointmentId <> I.AppointmentId
      AND A.AppointmentStatus <> 'Canceled'
      AND I.StartDateTime < A.EndDateTime
      AND I.EndDateTime > A.StartDateTime
  )
  BEGIN
    RAISERROR('Error: Veterinarian already booked for this time slot.', 16, 1);
    ROLLBACK TRANSACTION;
  END
END;
```

Purpose:
Automatically enforces the clinic's scheduling policy by blocking overlapping appointments for the same veterinarian. This ensures that invalid bookings cannot be inserted into the system.

b) trg_UpdateScheduleStatus

This trigger marks a veterinarian's schedule as "Busy" when a new appointment is created.

```
CREATE TRIGGER trg_UpdateScheduleStatus
ON Appointment
AFTER INSERT
AS
BEGIN
  UPDATE S
  SET S.ScheduleStatus = 'Busy'
  FROM Schedule S
  JOIN inserted I ON S.VetId = I.VetId
  WHERE S.ScheduleDate = CAST(I.StartDateTime AS DATE);
END;
```

Purpose:
Keeps the Schedule table synchronized with real appointments. When a veterinarian receives an appointment on a specific day, the system automatically updates their availability.

c) trg_AddReminderNotification

This trigger generates an automatic reminder notification whenever a new appointment is created.

```
CREATE TRIGGER trg_AddReminderNotification
ON Appointment
AFTER INSERT
AS
BEGIN
  INSERT INTO Notification (AppointmentId, NotificationType, SendDate, Channel,
NotificationStatus)
  SELECT
```

```
    I.AppointmentId,
    'Reminder',
    DATEADD(DAY, -1, I.StartDateTime),
    'Email',
    'Pending'
  FROM inserted I;
END;
```

Purpose:
Automatically schedules reminder messages for clients, ensuring they are notified before their appointments. This supports customer service and reduces no-shows.

3.6 Indexes

To optimize query performance and improve the responsiveness of the database, several non-clustered indexes were created. These indexes accelerate common search operations such as looking up appointments by veterinarian, retrieving owner animals, and querying medical histories.

Indexes were added **after table creation**, because SQL Server requires the table to exist before an index can be defined.

a) Purpose of Indexes

Indexes are used to:

- speed up SELECT queries
- improve JOIN performance
- optimize lookups in foreign key relationships
- reduce table scan operations
- improve stored procedure and view execution times

In this project, indexes were added on the most frequently searched and JOINed columns.

b) Index List and Justification

Index Name	Table	Column(s)	Purpose
IX_UserAccount_RoleId	UserAccount	RoleId	Speeds up filtering of users by role.
IX_Animal_OwnerId	Animal	OwnerId	Improves lookups of animals by owner.
IX_Schedule_VetId	Schedule	VetId	Optimizes vet schedule searches.
IX_Appointment_VetId	Appointment	VetId	Speeds up all vet-related appointment queries.
IX_Appointment_AnimalId	Appointment	AnimalId	Accelerates appointment lookups for animals.
IX_Appointment_StartDateTime	Appointment	StartDateTime	Improves date-based queries and reporting.
IX_MedicalHistory_AnimalId	MedicalHistory	AnimalId	Speeds retrieval of medical records by animal.

Index Name	Table	Column(s)	Purpose
IX_MedicalHistory_VetId	MedicalHistory	VetId	Useful for reviewing records created by a specific vet.
IX_Notification_AppointmentId	Notification	AppointmentId	Optimizes notification lookups for appointments.

c) Example Index Definition

CREATE INDEX IX_Appointment_StartDateTime

ON Appointment(StartDateTime);

Purpose:
This index improves the performance of monthly reports, daily agendas, and date-based filters used in the application.

3.7 Testing & Validation

All database objects were tested successfully using **SQL Server Management Studio (SSMS)**. Sample tests included:

- SELECT * FROM MonthlyReport;
- SELECT * FROM DailyAgenda;
- EXEC sp_AddAppointment ... (to test double-booking validation)
- EXEC sp_UpdateAppointmentStatus ...
- EXEC sp_GetVetAgenda ...

The test results confirmed:

- Data integrity was maintained.
- Views and stored procedures returned accurate, expected results.
- Constraints correctly prevented invalid or duplicate data.

All database objects were tested successfully using **SQL Server Management Studio (SSMS)**. Sample tests included:

- SELECT * FROM MonthlyReport;
- SELECT * FROM DailyAgenda;
- EXEC sp_AddAppointment ... (to test double-booking validation)
- EXEC sp_UpdateAppointmentStatus ...
- EXEC sp_GetVetAgenda ...

The test results confirmed:

- Data integrity was maintained.
- Views and stored procedures returned accurate, expected results.
- Constraints correctly prevented invalid or duplicate data.

3.8 Summary and Evaluation of Database Implementation

The implementation of the **AnimalCareDB** in **Microsoft SQL Server 2022** successfully translates the logical database design into a functioning physical database. **All tables, relationships, constraints, views, and stored procedures** operate as intended, ensuring accurate data handling, efficient reporting, and a strong foundation for the next development stage (**the ASP.NET Core MVC web application**). Testing confirmed that referential integrity, domain constraints, and business logic were all correctly enforced, validating the reliability of the database implementation.

In addition, the implemented **triggers** automate essential business rules such as preventing double-booking, updating schedule availability, and generating reminder notifications. These automated processes increase reliability and reduce the need for manual intervention.

The creation of **indexes** further enhances system performance by optimizing search operations, JOIN conditions, and reporting queries. Testing confirmed that referential integrity, domain constraints, triggers, and stored procedures all behaved correctly, validating the overall robustness and efficiency of the database implementation.